Retail E-Commerce Sales Analytics Pipeline

Project Report — Databricks · PySpark · Delta Lake · Medallion Architecture

Celebal Technologies — CEI Intern Assignment

Name: Jasmeet Kaur | **Roll No.:** 11233001

Maharishi Markandeshwar (Deemed to be University), Mullana

---

1. Objective / What Was Required

The assignment required building a production-style retail analytics pipeline for an e-commerce company using Databricks, PySpark, and Delta Lake, following the Medallion Architecture (Bronze → Silver → Gold). The source data included historical batch CSVs and daily incremental files, deliberately made "dirty" — containing null customer/product IDs, duplicate orders, invalid dates, text embedded in numeric columns, updated customer/product attributes, late-arriving orders, and a schema change (a new **coupon_code** column) introduced on Day 3 of the incremental feed.

The pipeline needed to: ingest raw files without loss, clean and standardize the data, maintain slowly changing Customer and Product dimensions using **SCD Type 2**, build a conformed fact table using surrogate-key point-in-time joins, and produce analytics-ready Gold tables for revenue, orders, customers, products, and stores.

2. Approach — Medallion Architecture

The pipeline was implemented as four sequential Databricks notebooks, each corresponding to one stage of the Medallion Architecture:

Stage	Notebook	Purpose
Bronze	Bronze_Layer_Setup	Raw, unmodified ingestion of batch + incremental files
Silver Stage 1	Silver_Stage1_Cleaning	Cleaning, type casting, deduplication, quarantine
Silver Stage 2	Silver_Stage2_SCD2	SCD Type 2 dimensions for Customer & Product
Gold	Gold_Layer_Fact_and_Analytics	Conformed fact table + 4 analytics tables

Catalog/Schema layout: **retail_demo** catalog with **raw**, **silver**, and **gold** schemas, and a Unity Catalog volume to stage the source files, exactly as suggested in the project guide.

3. What Was Done

3.1 Bronze Layer — Raw Ingestion

- Loaded all 4 historical batch files (orders, customers, products, stores) into Bronze Delta tables, reading every column as a string to avoid failures on dirty data.
- Loaded all 3 incremental/CDC file types (orders, customers CDC, products CDC) using **Auto Loader** (cloudFiles), which tracks already-processed files automatically and evolves schema without manual intervention.
- Added mandatory audit columns to every table: **source_file**, **ingestion_ts**, and **load_type** ("batch" / "incremental").

- Verified that the Day-3 schema change (new **coupon_code** column) was picked up automatically via Auto Loader's schema evolution, without any manual schema edits.

3.2 Silver Stage 1 — Cleaning, Casting, Deduplication

- **Orders:** Combined batch + incremental orders; cleaned unit_price using regex to strip non-numeric text (e.g. “unknown”) before casting to double; converted order_ts to a proper timestamp; identified and quarantined rows with null essential fields; deduplicated on order_id keeping the latest ingestion_ts.
- **Customers:** Parsed signup_date (invalid formats became NULL), replaced null city/segment with “Unknown”, quarantined rows missing customer_id/name, deduplicated on customer_id.
- **Products:** Same cleaning pattern — unit_price anomalies converted to NULL, null category defaulted to “Unknown”, deduplicated on product_id.
- **Stores:** Deduplicated on store_id to produce dim_store.
- Every rejected record was preserved in a dedicated **quarantine_*** table (quarantine_orders, quarantine_customers, quarantine_products) — nothing was silently dropped.

3.3 Silver Stage 2 — SCD Type 2 Dimensions

- Built dim_customer_scd2 and dim_product_scd2, each carrying surrogate_key, effective_start_date, effective_end_date, is_current, and a hash_value fingerprint of the tracked attributes.
- Performed an initial load from the Silver Stage 1 clean tables (every row versioned from 1900-01-01 to 9999-12-31, is_current = true).
- Processed each day's CDC file in chronological order using a SHA-256 hash of tracked attributes to detect real changes, applying the classic **2-step MERGE pattern**: (1) expire the old active row if the hash changed, (2) insert a new active row with a fresh surrogate_key.
- Verified exactly one “current” row per business key at all times, and inspected a sample customer's full change history to confirm historical tracking works correctly.

3.4 Gold Layer — Fact Table & Analytics

- Built **fact_orders** by resolving surrogate keys (customer_sk, product_sk, store_sk) from the SCD2 dimensions via a **point-in-time join** — the order date must fall between the dimension row's effective_start_date and effective_end_date — correctly handling late-arriving orders.
- Built 4 Gold analytics tables aggregated from fact_orders: gold_daily_sales, gold_category_sales, gold_segment_sales, and gold_region_sales.
- Demonstrated Delta Lake features: table history via DESCRIBE HISTORY, and time travel via SELECT ... VERSION AS OF 0.

4. Real-World Data Challenges Handled

Challenge	How It Was Handled
Null customer/product IDs	Identified and routed to quarantine_* tables instead of being dropped or crashing the pipeline.
Duplicate orders/records	row_number() over Window.partitionBy(business_key).orderBy(ingestion_ts desc) — keeps only the la
Invalid dates (e.g. 31-31-2025)	try_to_date() based parsing; unparseable dates become NULL and are handled downstream rather tha
Text inside numeric columns	Regex extraction of numeric characters from unit_price before casting to double.
Updated customer/product attributes	SCD Type 2 hash-based change detection with the 2-step expire-and-insert MERGE pattern.
Late-arriving orders	Point-in-time join resolves the correct historical dimension version regardless of when the order is proc
Schema change (coupon_code, Day 3)	Auto Loader schema evolution automatically picked up the new column with no manual schema chang

5. Key Techniques & Approaches Used

- **Schema-on-read in Bronze:** all columns read as strings to guarantee ingestion never fails on dirty data; casting deferred to Silver.
- **Auto Loader (cloudFiles)** for incremental ingestion — automatic file-tracking (no manual state management) and automatic schema evolution.
- **Window functions** for both deduplication (partitioned by business key, ordered by ingestion_ts) and surrogate key generation (global ordering for uniqueness).
- **SHA-256 hashing** of concatenated tracked attributes to detect real changes in SCD2, avoiding column-by-column comparison.
- **Two-step MERGE pattern** (expire, then insert) to correctly implement SCD Type 2, since a single MERGE cannot both update and insert for the same matched key.
- **Point-in-time join** (business key + date-range containment) to resolve the correct historical dimension version for every fact row, instead of joining on the natural key alone.
- **Surrogate keys as foreign keys** in the fact table rather than natural business keys — standard star-schema practice that also speeds up downstream Gold aggregations.
- **Quarantine pattern** for data quality — bad records are preserved and inspectable rather than silently dropped.

6. Final Deliverables (18 Tables)

Schema	Tables
raw	bronze_orders, bronze_orders_incremental, bronze_customers, bronze_customers_cdc, bronze_products, bronze_products_cdc, bronze_stores
silver	silver1_orders_clean, silver1_customers_clean, silver1_products_clean, dim_customer_scd2, dim_product_scd2, dim_store
gold	fact_orders, gold_daily_sales, gold_category_sales, gold_segment_sales, gold_region_sales

7. Results / Verification

All 18 expected deliverable tables were successfully populated. Row-count checks were run after every stage to confirm no unexpected data loss (e.g. 17,344 clean orders out of 18,315 raw rows, with 660 rows correctly quarantined). A sample customer's SCD2 history was inspected end-to-end to confirm that attribute changes (e.g. a city/segment update) correctly closed the old record and opened a new active version. DESCRIBE HISTORY and a VERSION AS OF time-travel query were run against the Gold fact table to demonstrate Delta Lake's built-in auditability.

8. Implementation Challenges Faced

- **Schema alignment across batch and incremental sources:** the coupon_code and ingest_date columns existed only in the incremental orders feed. Before union, these columns had to be explicitly added as NULL to the batch DataFrame so unionByName would not fail.
- **Idempotent SCD2 merges:** care was needed to ensure that re-running a day's CDC merge would not duplicate rows or re-expire already-closed records — solved by always filtering on is_current = true when matching and by comparing hashes rather than blindly inserting.
- **Surrogate key uniqueness vs. partitioning:** generating a single global sequence of surrogate keys required an unpartitioned Window.orderBy(), which triggers a Spark performance warning (WindowExpression: No Partition Defined). At this data volume (a few thousand rows) the impact is negligible; the alternative (partitioned numbering) would have produced colliding/duplicate keys, which is incorrect for a star-schema design.
- **Late-arriving and out-of-order CDC records:** processing each day's CDC file strictly in chronological order was necessary so that effective_start_date / effective_end_date boundaries remained consistent and no record was expired before its replacement existed.
- **Point-in-time correctness:** a naive join on the natural business key alone would have picked an arbitrary (often the wrong) historical version of a dimension row; the date-range containment condition was essential for correct historical reporting.

9. Key Learnings

This assignment reinforced several core data-engineering concepts that are directly applicable to real production pipelines:

- Why the Medallion Architecture separates concerns cleanly: Bronze for immutability and replay-ability, Silver for enforced data quality, and Gold for fast, dashboard-ready consumption.
- How Auto Loader removes the operational burden of manually tracking which incremental files have already been processed, and how it gracefully handles schema drift.
- Why SCD Type 2 is necessary whenever historical accuracy matters — e.g. “what segment was this customer in when they placed this order” cannot be answered correctly with a simple overwrite-in-place dimension.
- The practical mechanics of implementing SCD Type 2 with Delta Lake's MERGE — specifically why it must be split into two MERGE statements (expire, then insert) rather than one.
- The importance of a quarantine pattern for data quality: rejecting bad records silently makes debugging and auditing very difficult, whereas preserving them in a dedicated table keeps the pipeline both robust and transparent.
- How Delta Lake's transaction log (DESCRIBE HISTORY) and time travel (VERSION AS OF / TIMESTAMP AS OF) provide built-in auditability without any extra tooling.

10. Conclusion

The pipeline successfully implements a complete, production-style Medallion Architecture on Databricks and Delta Lake — from raw, immutable ingestion through cleaned/deduplicated Silver tables, historically-accurate SCD Type 2 dimensions, and a conformed fact table, up to dashboard-ready Gold aggregates. All dirty-data scenarios specified in the assignment (nulls, duplicates, invalid dates, embedded text, mid-stream schema changes, and late-arriving data) were explicitly identified and handled rather than ignored, and the required Delta Lake features (transaction history and time travel) were demonstrated.